Module 10: Graphs

CPE010 DATA STRUCTURES AND ALGORITHMS

Learning about Graphs and Graph Traversal Methods in C++

Prepared by Engr. R.M. Richard

Intended Learning Outcomes (ILOs)

After this module, the student should be able to:

- Create C++ code for graph implementation utilizing **adjacency matrix and adjacency list**.
- Create C++ code for implementing graph traversal algorithms such as **Breadth-First and Depth-First Search**.
- Demonstrate an understanding of **graph implementation, operations and traversal methods**.

Contents

The discussion today will cover the following topics:

- Graph Theory (Terminologies and Initial Implementation)
- Graph Traversal Algorithms
  - Breadth-First Search (BFS)
  - Depth-First Search (DFS)

What is a Graph?

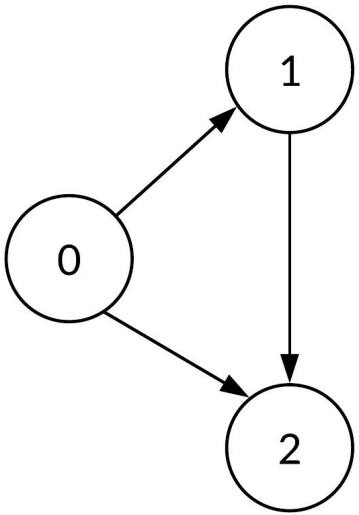
A graph is a **non-linear kind of data structure** made up of nodes or vertices and edges. The **edges** connect any two nodes in the graph, and the nodes are also known as **vertices**.

Types of Graphs

Two types of Graphs:

- **Undirected** Graph
- **Directed** Graph (digraph)

Directed Graph



Undirected Graph

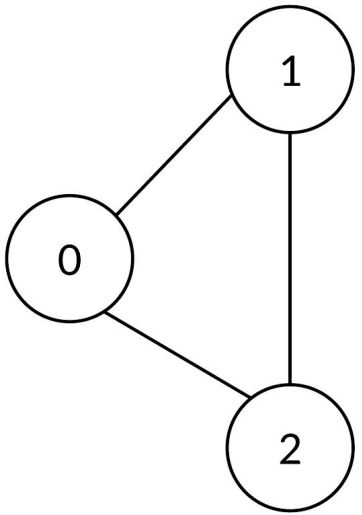
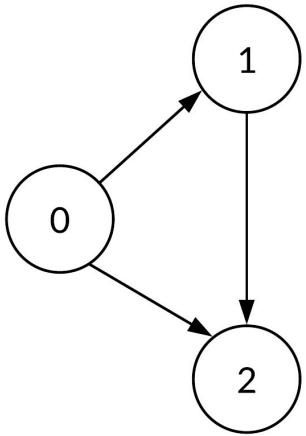


Image Source: CodePath Cliffnotes

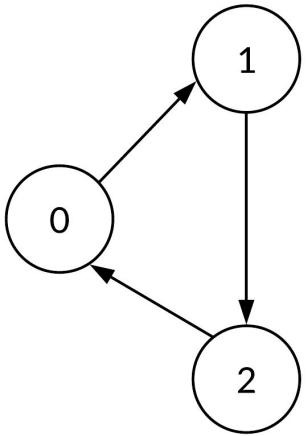
Paths

- A *path* of length *k* from *u* to *u'* is a sequence of vertices $\langle v_0, v_1, ..., v_k \rangle$ with $u = v_0$, $u' = v_k$, and $(v_{i-1}, v_i) \in E$.
- If a path *p* from *u* to *u'* exists, then *u'* is *reachable* from *u* (denoted $u \rightsquigarrow u'$ if *G* is a directed graph).
- The path is *simple* if all the vertices are distinct.
  - A *subpath* is a contiguous subsequence $\langle v_i, v_{i+1}, ..., v_j \rangle$ with $0 \le i \le j \le k$.
  - A *cycle* is a path with $v_0 = v_k$ (and is also simple if all the vertices except the end points are distinct). An *acyclic graph* is a graph with no cycles.

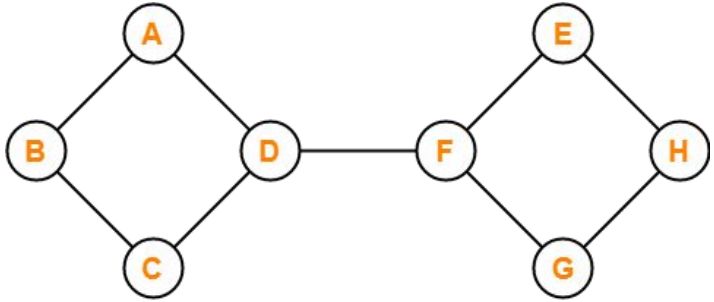
Acyclic Graph



Cyclic Graph



Other Terminologies



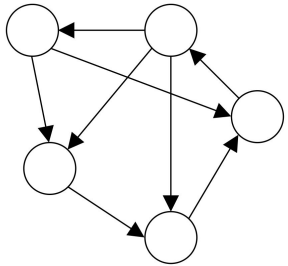
Example of Connected Graph

Connected Graph

- In an undirected graph, a connected component is a subset of vertices that are all reachable from each other. The graph is connected if it contains exactly one connected component, i.e. every vertex is reachable from every other.
- In a directed graph, a strongly connected component is a subset of mutually reachable vertices, i.e. there is a path between any two vertices in the set.

A **forest** is an undirected acyclic graph. If it is also connected, then it is a tree.

A **directed acyclic graph** is known as a DAG.

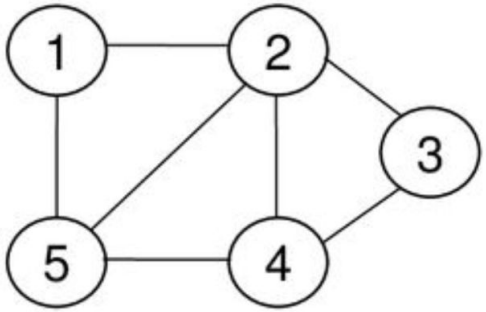


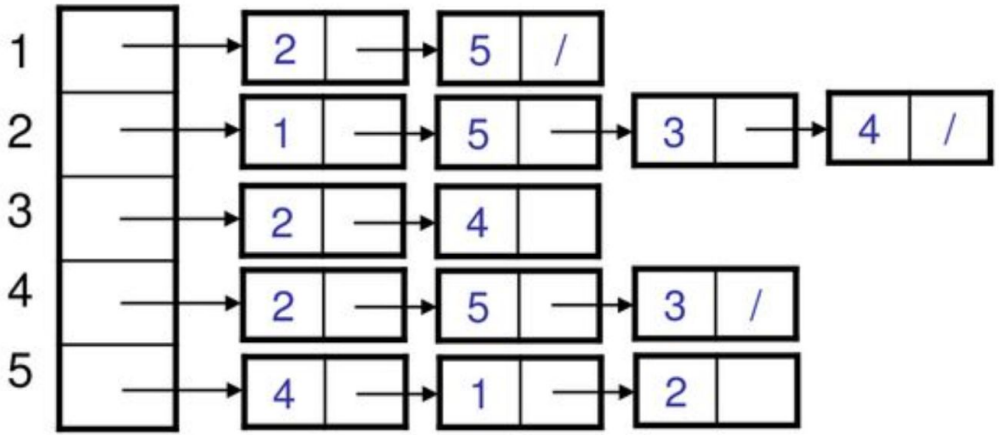
Graph Representation

A graph can be represented through either an **adjacency list** or an **adjacency matrix**.

- The **adjacency list** uses an array of lists. The size is equal to the number of nodes. A single index array[i] represents the lists of nodes adjacent to the *i*th node.
- The **adjacency matrix** is a 2D matrix array with a size of V x V where V is the number of nodes in a graph. A slot matrix[i][j] = 1 indicates that there is an edge from node *i* to node *j*.

Adjacency List Representation of G = (V, E)





- Has |V| lists for each vertex V.
- Adjacency exists if the node is connected to the list.
- Can be **used for both directed and undirected graph**.

Source: Egbert Ryan, 2019.

Properties of Adjacency List Representation

Sum of the lengths of all the adjacency lists

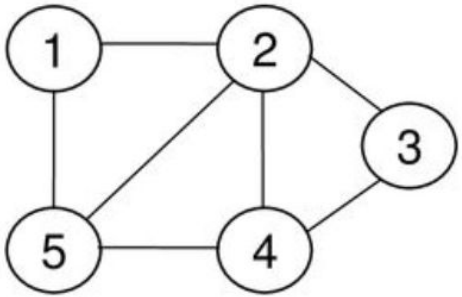
- **Directed graph**: |E| due to the edge pair (u,v) appearing only once in u's list.
- **Undirected graph**: 2|E| because u and v appear in each other's adjacency lists – edge (u, v) appears twice.

Preferred when the graph is **sparse**, meaning |E| is < $|V|^2$

Disadvantage: No quick way to determine whether there is an edge between (u, v).

Memory required is |V+E|

Adjacency Matrix Representation of G = (V,E)



	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Assume vertices are numbered 1, 2, …, |V|.

The representation consists of a matrix $A_{|V|x|V|}$

$$a_{ij} = \begin{cases} 1 & if (i,j) \in E \\ 0 & otherwise \end{cases}$$

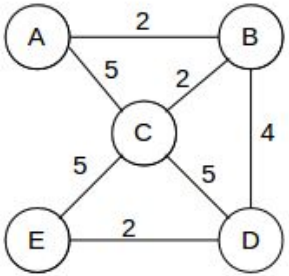
Properties of Adjacency Matrix Representation

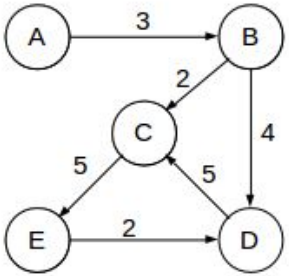
Preferred when the graph is **dense** such that |E| is close to $|V|^2$

Also, preferred when there is a need to **quickly determine presence of an edge** between two vertices.

Memory required is V^2

Weighted Graphs





These are graphs wherein each edge has an associated weight w(u,v) such that:

W: E -> R, weight function

Storing the weight can be done in both graph representation methods:

- In adjacency lists, store **w(u, v)** along with vertex v in u's adjacency list.
- In adjacency matrix, store **w(u,v)** at location (u, v) in the matrix.

Graph Theory to Implementation
Demonstration of how to make a graph in C++

Graph Traversal Algorithms

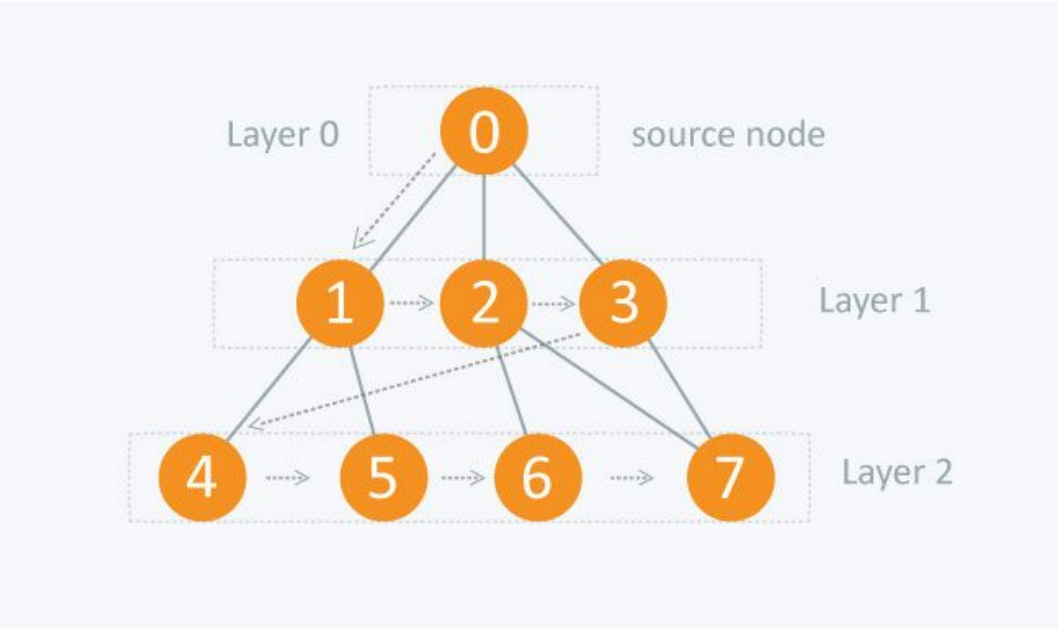
We will look at two traversal algorithms:

- Breadth-First Search
- Depth-First Search

Breadth-First Search

Definition: BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layerwise thus exploring the neighbour nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbour nodes.

BFS Continuation



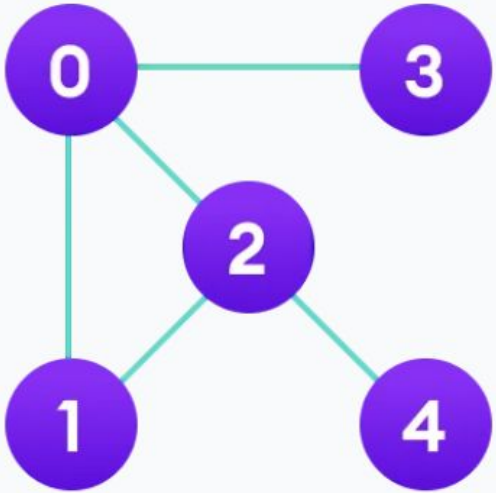
As the name BFS suggests, you are required to traverse the graph breadthwise as follows:

1. First move horizontally and visit all the nodes of the current layer
2. Move to the next layer

BFS Algorithm

The algorithm works as follows:

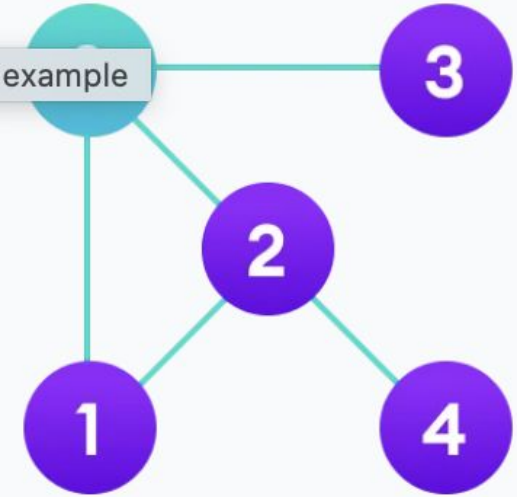
1. Start by putting any one of the graph's vertices at the back of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.



Visited

Queue

FRONT



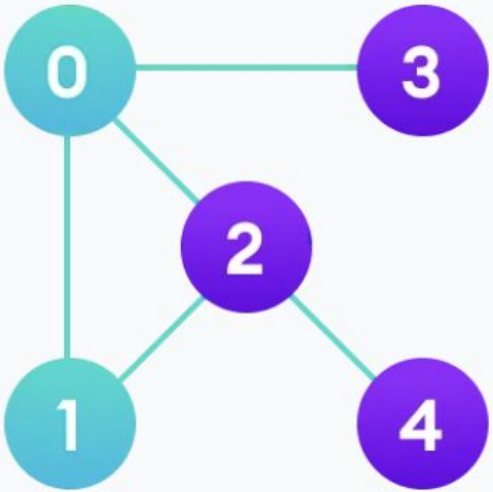
0				

Visited

1	2	3		

Queue

FRONT



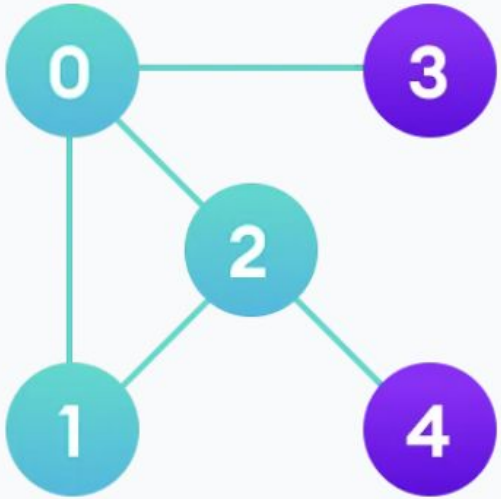
0	1			

Visited

2	3			

Queue

FRONT



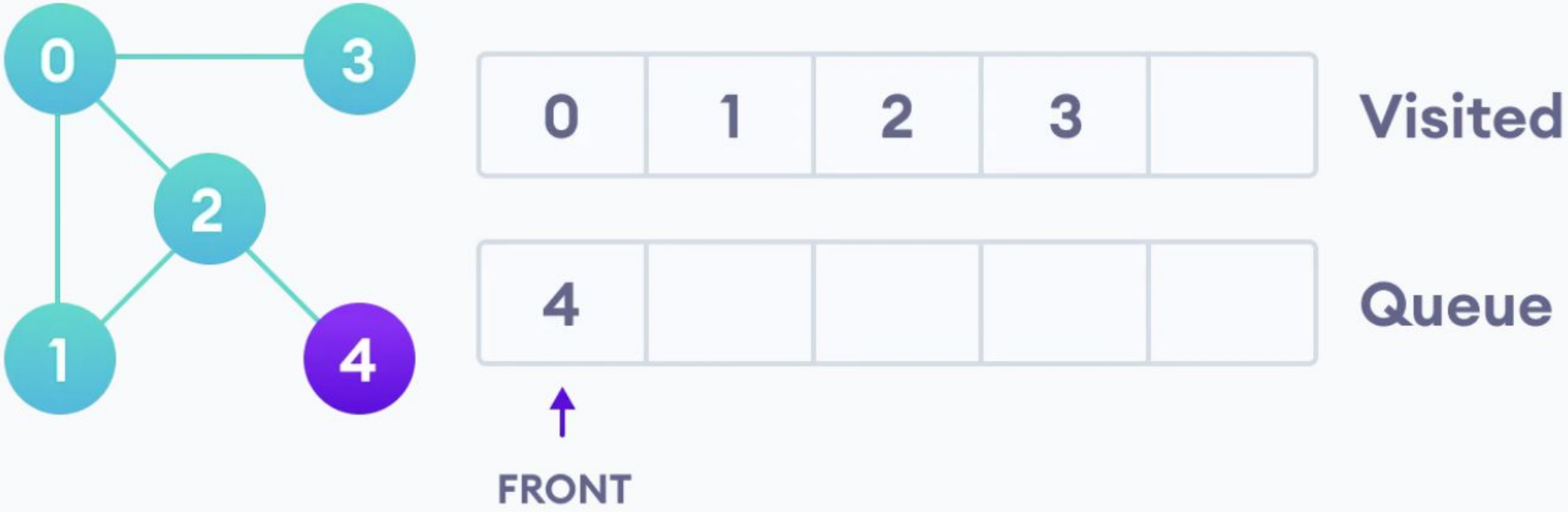
0	1	2		

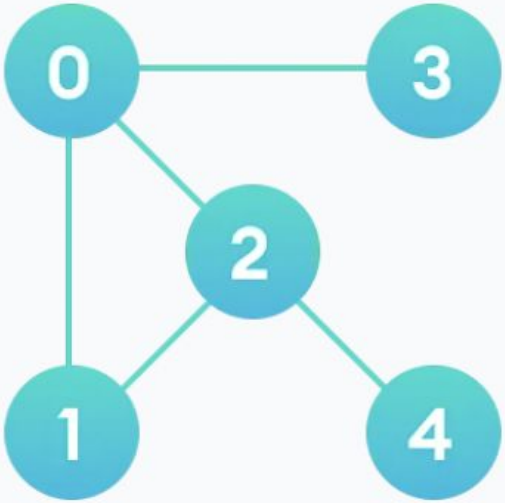
Visited

3	4			

Queue

FRONT


0
3
2
1
4
0
1
2
3
Visited
4
Queue
FRONT



0	1	2	3	4	Visited
					Queue

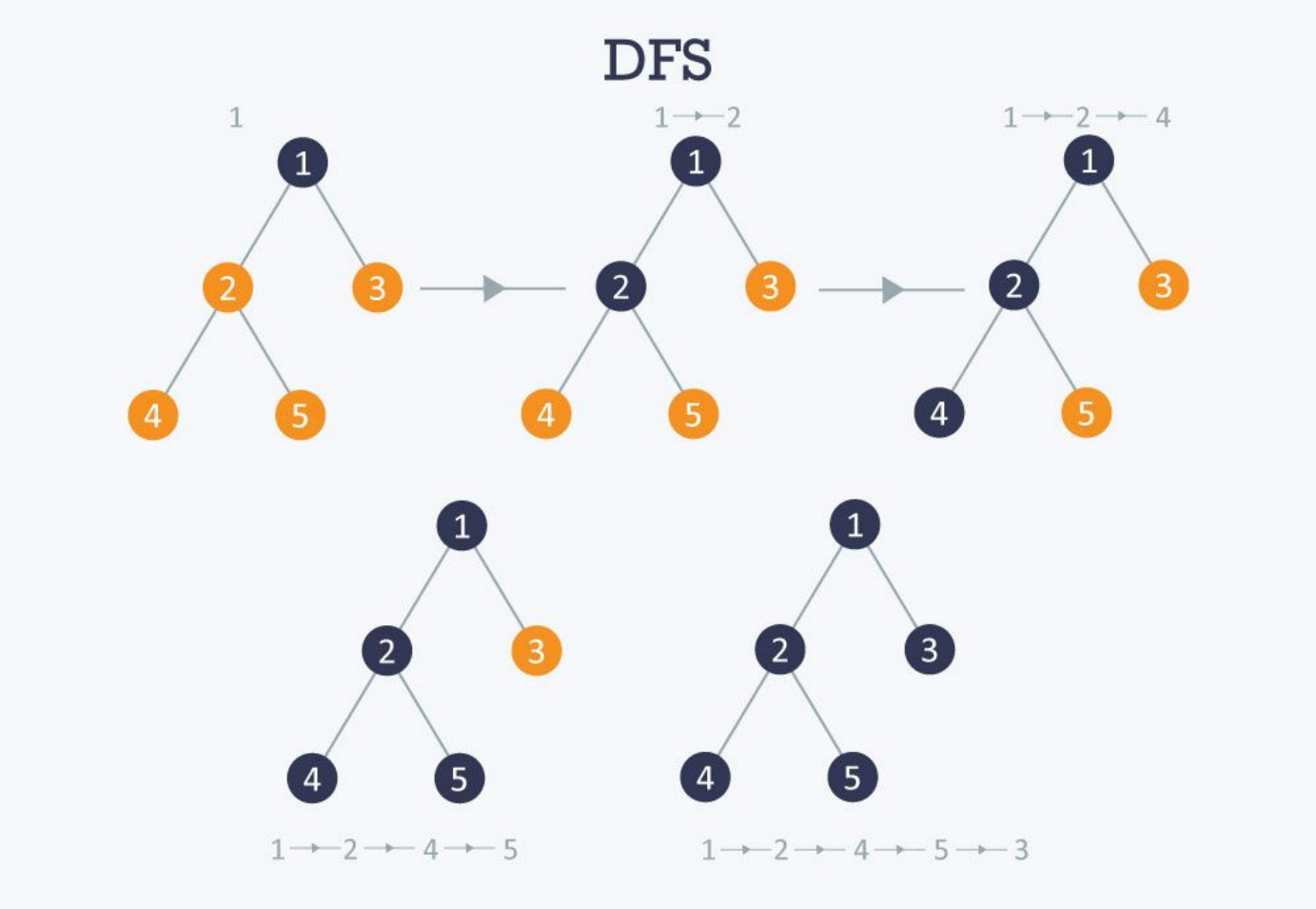
FRONT

Breadth-First Search Implementation

Graph Traversal Algorithm in C++

Depth-First Search

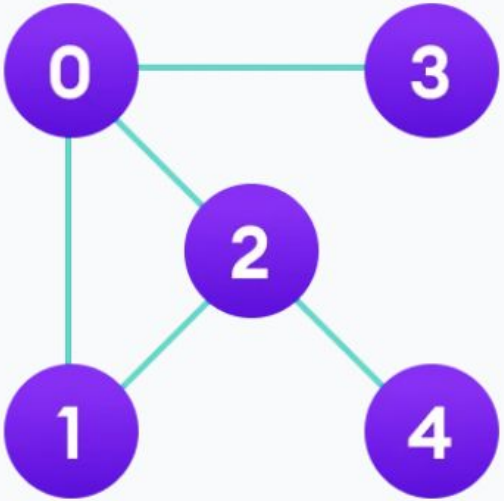
Definition: The DFS algorithm is a recursive algorithm that uses the idea of backtracking. It involves exhaustive searches of all the nodes by going ahead, if possible, else by backtracking. Here, the word backtrack means that when you are moving forward and there are no more nodes along the current path, you move backwards on the same path to find nodes to traverse. All the nodes will be visited on the current path till all the unvisited nodes have been traversed after which the next path will be selected.


DFS
1
1 → 2
1 → 2 → 4
1 → 2 → 4 → 5
1 → 2 → 4 → 5 → 3

DFS Algorithm

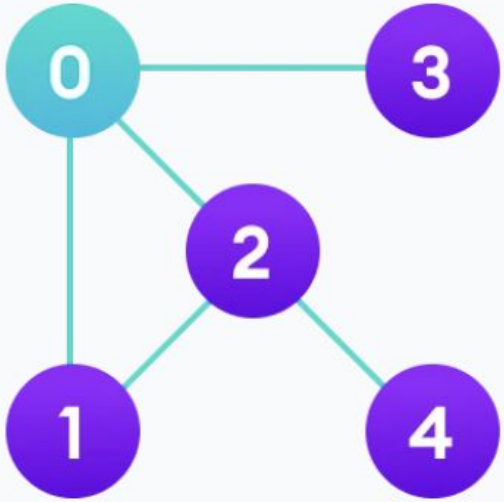
The DFS algorithm works as follows:

1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.



Visited

Stack

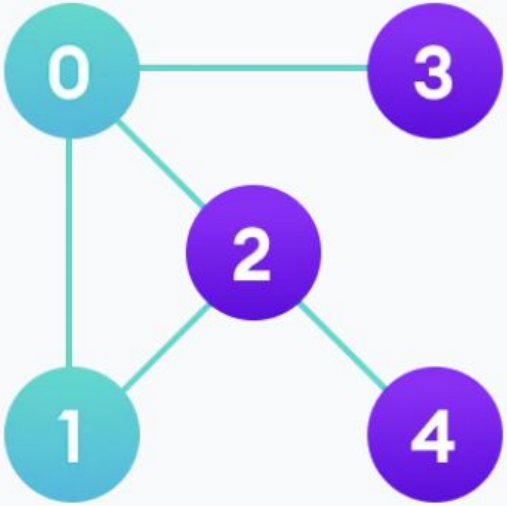


0				

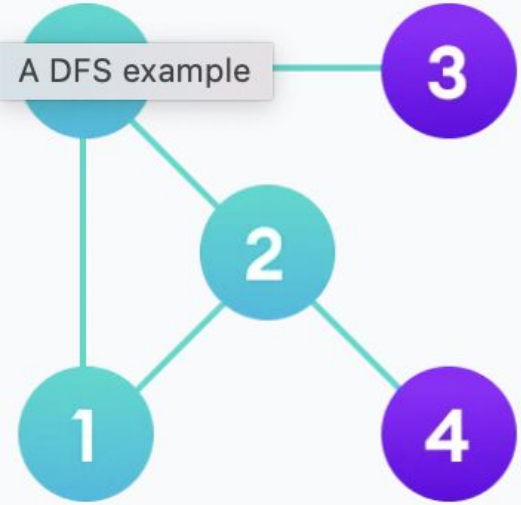
Visited

1	2	3		

Stack

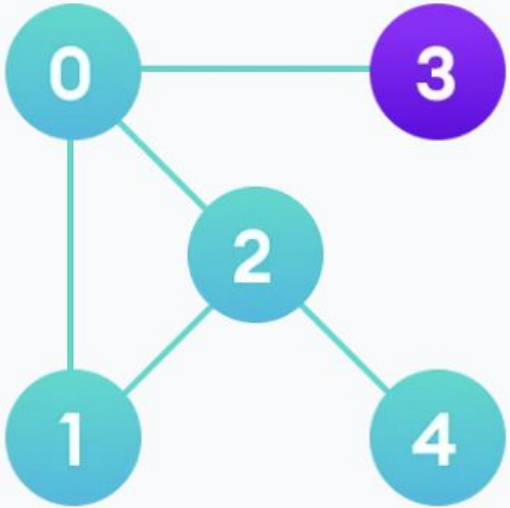


0	1				Visited
2	3				Stack



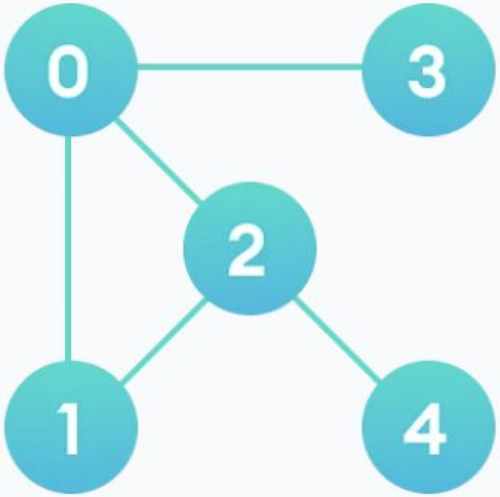
0	1	2			Visited

4	3				Stack



0	1	2	4		Visited

3					Stack



0	1	2	4	3	Visited
					Stack

Depth-First Search Implementation

Graph Traversal Algorithm in C++

Concept Check!

Summary

Today, the following topics were covered:

- Definition of Graphs
- Graph implementation methods
- Traversal methods
- Implementation of traversal methods

Reminders and Requirements for Module 10

- Submit Hands-on Activity 10.1 on/before the given deadline.
- Late policies still apply.
- If you have questions, you may contact me through our official communication channels. However, please expect delays outside class hours.

References

https://www.hackerearth.com/practice/algorithms/graphs/graph-representation/tutorial/

https://www.simplilearn.com/tutorials/data-structure-tutorial/graphs-in-data-structure

https://www.educative.io/answers/what-is-a-graph-data-structure

http://ycpcs.github.io/cs360-spring2015/lectures/lecture15.html

http://ycpcs.github.io/cs360-spring2015/lectures/lecture16.html

http://ycpcs.github.io/cs360-spring2015/lectures/lecture17.html

https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/GraphTraversal.html#:~:text=Graph%20traversal%20algorithms%20typically%20begin,the%20graph%20is%20not%20connected.